

COMSATS University Islamabad (CUI)

Project Proposal

for

FalconScan (AI-powered APK Vulnerability Scanner)

Version 1.0

By

Mohamed Ahsan **CIIT/FA22 -BCT-001/ISB**

Abdullah **CIIT/FA22-BCT-004 /ISB**

Supervisor

Dr. Farhana Jabeen

Bachelor of Science in Cyber Security (2022-2026)

Table of Contents

Abstract 2
1. Introduction 3
2. Problem Statement 4
3. Problem Solution/Objectives of the Proposed System 4
   3.1 Objectives 5
4. Related System Analysis/Literature Review 6
5. Vision Statement 6
6. Scope 7
7. Modules 8
   7.1 Module 1: APK Upload and Management 8
   7.2 Module 2: Static Analysis Engine 8
   7.3 Module 3: Dynamic Analysis Engine 8
   7.4 Module 4: LLM-Driven Vulnerability Analysis 8
   7.5 Module 5: Reporting and Visualization 8
   7.6 Module 6: User Interface and Access Control 9
8. System Limitations/Constraints 9
9. Data Gathering Approach 10
10. Tools and Technologies 10
11. Project Stakeholders and Roles 10
12. Module based Work Division 11
13. WBS and Gantt Chart 12
14. Mockups 14
15. References 17
16. Plagiarism Report 19

Project Category:

☐ **A-**Desktop Application/Information System	☑ **B-**Web Application/Web Application based Information System	
☐ **C-**Problem Solving and Artificial Intelligence	☐ **D-**Simulation and Modeling	☐ **E-**Smartphone Application
☐ **F-**Smartphone Game	☐ **G-**Networks	☐ **H-**Image Processing
☐ Other (specify category) _____		

Abstract

Traditional Android application vulnerability detection methods are often manual-intensive, time-consuming, and inefficient, making them inadequate for addressing modern cybersecurity challenges. Existing tools such as MobSF and VirusTotal rely heavily on rule-based static checks or limited dynamic tracing, failing to effectively detect obfuscated or unknown threats. These systems lack intelligent interpretation of runtime behaviors and require significant human effort for result analysis.

This project presents FalconScan, an AI-Based APK Vulnerability Assessment Tool that automates the identification and classification of security threats in Android applications using a hybrid analysis pipeline integrating static code inspection, dynamic behavior monitoring, and LLM-powered reasoning. The system introduces a novel workflow where structured features extracted from both static and dynamic phases are fed into a pre-trained Large Language Model (LLM) for intelligent classification and explainable decision-making.

FalconScan's core innovation lies in its Structured Feature Template and Chain-of-Thought (CoT) reasoning engine, which enables the LLM to interpret complex behavioral patterns and justify its classification decisions in human-readable form. Additionally, the system supports continuous learning through feedback loops and periodic model updates, ensuring adaptability to evolving malware tactics.

By leveraging AI-driven automation, FalconScan significantly improves detection accuracy, reduces manual intervention, and provides actionable insights for developers and security analysts. It represents a scalable and intelligent solution for enhancing mobile app security in both research and production environments.

1. Introduction

The rapid proliferation of Android applications has expanded the attack surface for cyber threats, necessitating more robust and automated security assessment mechanisms. Despite the availability of various tools like MobSF and VirusTotal, traditional approaches to Android vulnerability detection remain largely fragmented — relying on either rule-based static analysis or limited dynamic instrumentation. These methods struggle with detecting sophisticated threats such as obfuscated payloads, reflective calls, and covert communication channels, often requiring manual validation and expert interpretation.

To address these limitations, this project introduces FalconScan, an intelligent hybrid vulnerability scanner designed to automate the detection of malicious behaviors in Android APKs. FalconScan integrates static code analysis, sandboxed dynamic execution, and LLM-based threat classification into a unified pipeline. Unlike conventional scanners, FalconScan does not rely solely on signature-based detection but leverages machine understanding to interpret both structural and behavioral indicators of compromise.

At the heart of FalconScan is an LLM-powered detection engine that analyzes structured features derived from static and dynamic analyses — including permissions, API usage, network activity, and system logs. Using Chain-of-Thought (CoT) reasoning, the model evaluates each APK step-by-step, identifying suspicious patterns and generating human-readable explanations for its findings. This interpretability enhances transparency and trust in the system's output.

FalconScan also incorporates a Model Context Protocol (MCP) to log and trace AI-driven insights, supporting auditability and enabling future retraining. The system executes apps in a controlled environment that observes real-time behaviors and correlates them with static artifacts to identify anomalies that may indicate malicious intent.

Designed primarily for educational and research use, FalconScan aims to reduce the burden on analysts by automating complex tasks, improving detection accuracy, and delivering comprehensive security reports.

Ultimately, FalconScan represents a paradigm shift in mobile application security assessment — transforming fragmented, rule-based tools into an intelligent, self-improving, and highly effective defense against modern Android threats.

2. Problem Statement

Cybersecurity analysts and developers often struggle to identify security flaws in Android applications before they are released to the public. Existing tools like MobSF and VirusTotal focus primarily on static analysis, which detects only known patterns and rule-based issues such as dangerous permissions or hardcoded secrets. However, many advanced threats use obfuscation and code injection techniques to bypass static detection, and traditional tools cannot effectively analyze such behavior. Dynamic analysis, while more insightful, usually requires manual setup, separate tools, and individual analysis of logs, network traffic, and runtime behavior. This fragmented approach is time-consuming, inconsistent, and unable to scale for large numbers of APKs. Moreover, current tools rarely include artificial intelligence to detect vulnerabilities, leaving a significant gap in mobile threat detection. As a result, malicious apps with new or hidden behaviors can go undetected, exposing end-users to privacy breaches, data theft, and other security risks. Our goal is to develop **an intelligent, hybrid APK vulnerability scanner that combines both static and dynamic analysis with AI-based detection** to solve this problem. The system will automate the scanning process, analyze behavioral patterns at runtime, and use machine learning to detect both known and unknown threats. By consolidating these techniques into one platform, the project will reduce manual effort, improve detection accuracy, and enhance the overall security assessment process for Android applications.

3. Problem Solution/Objectives of the Proposed System

The proposed APK Vulnerability Scanner aims to deliver a smart, hybrid solution that automates both static and dynamic analysis of Android applications. This system is intended to help cybersecurity professionals, developers, and testers identify security issues in APKs more efficiently and intelligently. The primary objective is to combine traditional rule-based analysis with AI-driven threat detection to uncover vulnerabilities. Unlike current tools that focus only on static signatures or offer limited runtime insights, our system will integrate behavioral monitoring during app execution. It will use emulation to capture runtime data such as file access, network communication, and API calls. These logs will be further analyzed using machine learning models trained on malware datasets to classify suspicious patterns and anomalies. The static analysis module will extract permissions, manifest configurations, code patterns, and third-party libraries. Meanwhile, the dynamic module will track the actual behavior of the app during execution to catch hidden logic or runtime threats.

One of the key goals is to develop a web-based interface where users can simply upload an APK and receive a full security assessment report. This report will include severity levels and AI predictions. The system will support continuous learning by periodically retraining the AI model with new malware samples. The tool will also maintain an internal vulnerability database for faster signature matching. Overall, the proposed system will provide a complete, automated, and intelligent solution for detecting Android app vulnerabilities — setting itself apart from traditional tools in terms of accuracy, efficiency, and innovation.

3.1 Objectives

- **BO-1**: Detect known APK vulnerabilities through automated static analysis with at least 90% accuracy.
- **BO-2**: Identify unknown threats using AI-based classification of static and dynamic features.
- **BO-3**: Automate the entire scanning workflow to reduce manual analysis time by 60%.
- **BO-4**: Provide detailed vulnerability reports with severity scores and remediation suggestions.
- **BO-5**: Track and analyze app behavior at runtime using an emulator for dynamic threat detection.
- **BO-6**: Integrate both static and dynamic analysis into a unified hybrid model for comprehensive security assessment.
- **BO-7**: Enable APK uploads and scan initiation via a user-friendly web interface.
- **BO-8**: Maintain a local database of known malicious signatures for fast rule-based detection.
- **BO-9**: Achieve at least 85% accuracy in AI-based malware classification using behavioral logs.
- **BO-10**: Reduce reliance on external tools by combining multiple functionalities in one platform.
- **BO-11**: Improve detection of obfuscated or disguised malicious behavior through AI analysis.
- **BO-12**: Support continuous learning by updating the AI model with new threat data periodically.
- **BO-13**: Generate comprehensive visual reports and logs that simplify decision-making for security analysts.

4. Related System Analysis/Literature Review

Several tools and platforms exist for analyzing APK files, focusing on either static or dynamic analysis. **MobSF (Mobile Security Framework)** is a widely used open-source tool that performs both static and limited dynamic analysis. However, its dynamic analysis depends on emulation and may fail to detect sophisticated, runtime-triggered threats. **VirusTotal**, which aggregates multiple antivirus engines, offers basic static and limited dynamic analysis using Droidbox, but lacks advanced behavioral insight or AI-based detection. **Quark Engine** uses rule-based static analysis but does not offer dynamic analysis or machine learning. **Droidbox** and **CuckooDroid** are dynamic analysis frameworks but require significant manual setup and don't integrate static analysis or AI. These systems either lack automation, real-time behavior detection, or AI-driven threat intelligence.

Our proposed project combines the strengths of these tools while addressing their limitations. It will offer a **fully automated hybrid system** that integrates both **static and dynamic analysis** and applies **AI models** to predict threats. By unifying rule-based scanning with intelligent behavior analysis, it bridges the gap left by existing tools and ensures more comprehensive APK vulnerability detection.

Application Name	Weakness	Proposed Project Solution
MobSF	Limited dynamic analysis.	Adds robust AI-based detection and deeper dynamic behavior capture.
VirusTotal	Offers basic static/dynamic scanning without custom logic or AI.	Implements custom logic, AI threat models, and enhanced dynamic tracing.
Quark Engine	Only static rule-based analysis; no dynamic or AI capability.	Combines static + dynamic analysis and adds AI for unknown threat detection.
Droidbox	Requires manual configuration; no static or AI integration.	Fully automated with combined static-dynamic scanning and intelligent analysis.
CuckooDroid	Complex setup; focuses only on dynamic analysis.	Automates analysis pipeline and integrates AI-based detection across both modes.

Table 1: Related System Analysis with proposed project solution

5. Vision Statement

For cybersecurity professionals, researchers, and app developers who need a powerful, automated, and intelligent system to detect both known and unknown vulnerabilities in Android applications. The FalconScan is a hybrid AI-powered APK vulnerability scanning system that performs in-depth static and dynamic analysis and generates comprehensive security reports with actionable insights.Unlike traditional tools like MobSF or VirusTotal that either lack deep dynamic analysis or AI capabilities. Our product integrates advanced behavioral analysis, automation, and AI-based threat intelligence into a single platform, offering a more accurate, scalable, and future-ready solution for mobile application security.

6. Scope

The scope of this project is to develop a fully standalone **AI-powered APK Vulnerability Scanner** that integrates both **static and dynamic analysis** techniques to identify security vulnerabilities in Android applications.

The static analysis module will reverse engineer APKs using decompilation techniques (using tools like JADX or apktool) and analyze AndroidManifest.xml, API calls, permissions, intent filters, and hardcoded credentials. The dynamic analysis module will deploy the APK in a custom sandbox or emulator (e.g., using AVD or Genymotion) and monitor behaviors such as network activity, file operations, service invocations, and abnormal runtime actions.

AI will be integrated through trained machine learning models to detect unusual behavioral patterns and predict unknown vulnerabilities based on features extracted during both static and dynamic phases. Training datasets will be prepared from open-source malware repositories and clean APKs to distinguish malicious traits. The system will provide a web-based GUI where users can upload APKs, initiate scans, and receive in-depth vulnerability reports along with risk scores and AI-based insights.

The tool will support scheduled and batch scans, maintain logs and historical data, and offer visualizations of dynamic behavior. It will also include features for customizable rules, export reporting, and user management for professional use.

7. Modules

7.1 Module 1: APK Upload and Management

- **FE-1:** Upload APK files via the web interface for scanning.
- **FE-2:** Store uploaded APKs with metadata (name, size, hash, upload date).
- **FE-3:** Allow users to browse scan history and reports.
- **FE-4:** Delete or archive previously scanned APKs.

7.2 Module 2: Static Analysis Engine

- **FE-1:** Decompile APKs using local tools (apktool, JADX).
- **FE-2:** Analyze `AndroidManifest.xml` for permission misuse and exposed components.
- **FE-3:** Scan source code for hardcoded secrets, risky API calls, and insecure configurations.
- **FE-4:** Extract and analyze strings, URLs, and embedded certificates.
- **FE-5:** Feature extraction for AI model (e.g., permissions, APIs, intent actions).

7.3 Module 3: Dynamic Analysis Engine

- **FE-1:** Deploy APKs in an isolated Android emulator (AVD/Genymotion).
- **FE-2:** Monitor runtime behavior (file access, network connections, service invocations).
- **FE-3:** Capture logs, system calls, and runtime exceptions.
- **FE-4:** Perform controlled input injection to simulate user interaction.
- **FE-5:** Extract behavior features for AI processing.

7.4 Module 4: LLM-Driven Vulnerability Analysis

- **FE-1:** Preprocess extracted static and dynamic features.
- **FE-2:** Classify APKs as benign or malicious using trained LLM model.
- **FE-3:** Display vulnerability scores and AI confidence levels.
- **FE-4:** Update model with new training data periodically (offline learning module).
- **FE-5**: Utilize the **Model Context Protocol (MCP)** to structure input features, standardize analysis context, and enable traceable decision paths from AI responses.

7.5 Module 5: Reporting and Visualization

- **FE-1:** Generate detailed vulnerability reports with categorized issues.
- **FE-2:** Visualize dynamic behaviors (e.g., system call graphs, network maps).
- **FE-3:** Allow export reports as PDF or JSON.
- **FE-4:** Summarize overall APK risk score and provide actionable suggestions.

7.6 Module 6: User Interface and Access Control

- **FE-1:** Web-based dashboard for students, researchers, and developers.
- **FE-2:** Login and registration system with role-based access (admin/user).
- **FE-3:** Provide access control for scanning, viewing reports, and deleting data.
- **FE-4:** Audit logs of actions performed by each user.

8. System Limitations/Constraints

- **LI-1:** The system will only support `.apk` files; apps developed for iOS or other platforms are not within the scope.

- **LI-2:** Dynamic analysis is limited to behavior observed within a controlled emulator environment and may not reflect advanced real-world evasion techniques.

- **LI-3:** AI detection accuracy depends on the quality and quantity of the training dataset; unknown threats may occasionally go undetected.

- **LI-4:** The tool cannot analyze obfuscated or encrypted APKs effectively without advanced unpacking methods, which are currently out of scope.

- **LI-5:** Dynamic analysis is resource-intensive and may require hardware with virtualization support and adequate RAM to run emulators smoothly.

- **LI-6:** Internet access will be disabled during analysis to prevent real malware from communicating with live command-and-control servers.

- **LI-7:** The solution is intended for educational and research purposes; it is not a replacement for professional-grade commercial mobile security tools.

- **LI-8:** Real-time malware detection on physical devices is not included; the scope is limited to offline analysis of uploaded APKs.

- **LI-9:** The system will not provide automated patching or remediation of detected vulnerabilities; it will only offer suggestions.

9. Data Gathering Approach

To gather accurate and relevant requirements for the proposed APK Vulnerability Scanner, a combination of interviews and literature review was used. Interviews were conducted with cybersecurity professionals and mobile app developers to understand the common vulnerabilities found in APKs and the limitations of existing tools. In addition, academic research papers, OWASP Mobile Top 10, and case studies were analyzed to identify the requirements for both static and dynamic analysis features. This multi-source approach ensured that the system design aligns with real-world needs and current research trends.

10. Tools and Technologies

	Tools	**Version**	**Rationale**
Tools and Technologies	Android Studio	Hedgehog	APK reverse engineering and analysis
	Genymotion	3.5.0	Android emulator for dynamic testing
	Jupyter Notebook	Latest	AI model training and experimentation
	SQLite	3.42.0	Lightweight database for storing results
	Git and Github	2.42.0	Version control
	Burp Suite	2023.11	For intercepting and analyzing APK traffic
	Apktool	2.9.3	Decompilation of APKs
	jadx	1.4.5	Source code analysis
	Technologies	**Version**	**Rationale**
	Python	3.11	Scripting and AI implementation
	TBD	TBD	AI model development
	MongoDB	8.0.0	Database
	TBD	TBD	Backend web service for scanner interface
	React	Latest	Front-end development
	ADB (Android Debug Bridge)	Latest	Dynamic APK interaction & automation

Table 2: Tools and technologies

11. Project Stakeholders and Roles

Project Sponsor	COMSATS University Islamabad, Islamabad Campus
Stakeholder	• Students' Name: Mohamed Ahsan (FA22-BCT-001) Abdullah (FA22-BCT-004) • Project Supervisor Name: Dr. Farhana Jabeen • Final Year Project Committee: Evaluation of project

Table 3: Project Stakeholders for Proposed Project

12. Module based Work Division

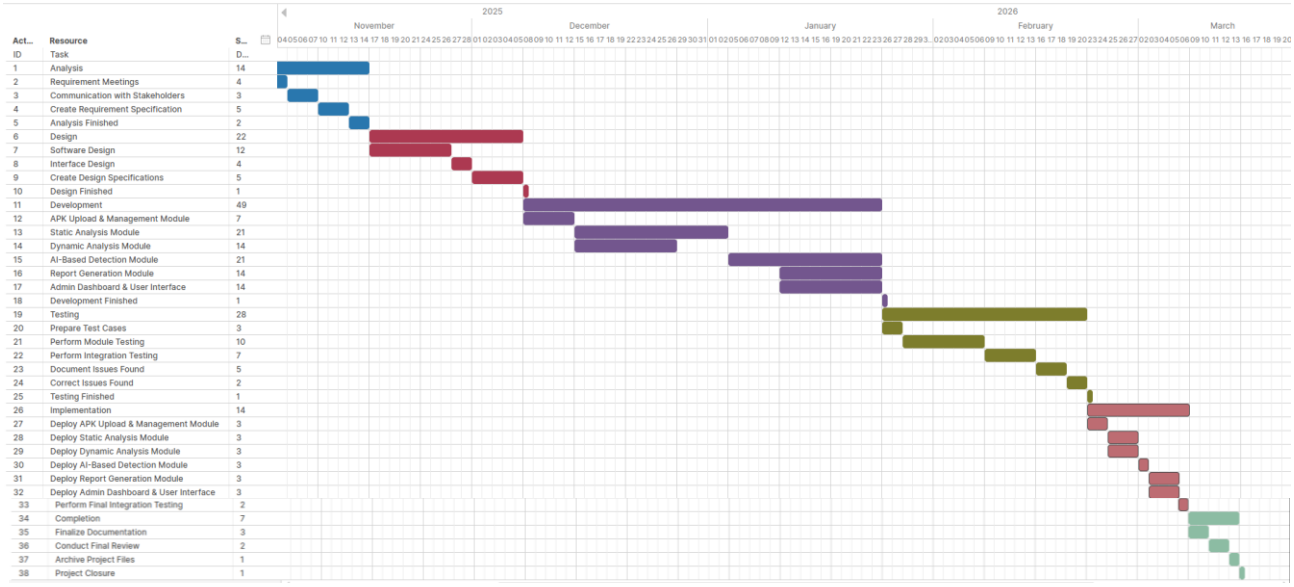
Student Name	Student Registration Number	Responsibility / Module / Feature
Mohamed Ahsan	FA22-BCT-001	Module 1: APK Upload & Management (uploading, organizing, managing scanned apps)
		Module 3: Dynamic Analysis (automated emulator setup, behavior capture, logs)
		Module 5: Report Generation (vulnerability summary, CVSS scoring, export PDF/HTML)
Abdullah	FA22-BCT-004	Module 2: Static Analysis (permissions, code patterns, rule-based detection)
		Module 4: AI-based Vulnerability Detection (dataset, training, classification)
		Module 6: Admin Dashboard & User Interface (frontend design, role management, status monitoring)

Table 4: Team Member Work Division for Proposed Project

13. WBS and Gantt Chart

ID	Task	Start Date	End Date	Duration (Days)	Resources
1	**Analysis**	01-11-2025	14-11-2025	14	Ahsan, Abdullah
2	Requirement Meetings	01-11-2025	04-11-2025	4	Ahsan, Abdullah
3	Communication with Stakeholders	05-11-2025	07-11-2025	3	Ahsan, Abdullah
4	Create Requirement Specification	08-11-2025	12-11-2025	5	Ahsan, Abdullah
5	Analysis Finished	13-11-2025	14-11-2025	2	Ahsan, Abdullah
6	**Design**	15-11-2025	06-12-2025	22	Ahsan, Abdullah
7	Software Design	15-11-2025	26-11-2025	12	Ahsan, Abdullah
8	Interface Design	27-11-2025	30-11-2025	4	Abdullah
9	Create Design Specifications	01-12-2025	05-12-2025	5	Ahsan, Abdullah
10	Design Finished	06-12-2025	06-12-2024	1	Ahsan, Abdullah
11	**Development**	07-12-2025	24-01-2026	49	Ahsan, Abdullah
12	APK Upload & Management Module	07-12-2025	13-12-2025	7	Ahsan
13	Static Analysis Module	14-12-2025	03-01-2026	21	Abdullah
14	Dynamic Analysis Module	14-12-2025	27-12-2025	14	Ahsan
15	AI-Based Detection Module	04-01-2026	24-01-2026	21	Abdullah
16	Report Generation Module	11-01-2026	24-01-2026	14	Ahsan
17	Admin Dashboard & User Interface	11-01-2026	24-01-2026	14	Abdullah
18	Development Finished	24-01-2026	24-01-2026	1	Ahsan, Abdullah
19	**Testing**	25-01-2026	21-02-2026	28	Ahsan, Abdullah
20	Prepare Test Cases	25-01-2026	27-01-2026	3	Ahsan, Abdullah
21	Perform Module Testing	28-01-2026	06-02-2026	10	Ahsan, Abdullah
22	Perform Integration Testing	07-02-2026	13-02-2026	7	Ahsan, Abdullah
23	Document Issues Found	14-02-2026	18-02-2026	5	Ahsan, Abdullah
24	Correct Issues Found	19-02-2026	20-02-2026	2	Ahsan, Abdullah
25	Testing Finished	21-02-2026	21-02-2026	1	Ahsan, Abdullah
26	**Implementation**	22-02-2026	07-03-2026	14	Ahsan, Abdullah
27	Deploy APK Upload & Management Module	22-02-2026	24-02-2026	3	Ahsan
28	Deploy Static Analysis Module	25-02-2026	27-02-2026	3	Abdullah
29	Deploy Dynamic Analysis Module	25-02-2026	27-02-2026	3	Ahsan
30	Deploy AI-Based Detection Module	28-02-2026	02-03-2026	3	Abdullah
31	Deploy Report Generation Module	03-03-2026	05-03-2026	3	Ahsan
32	Deploy Admin Dashboard & User Interface	03-03-2026	05-03-2026	3	Abdullah
33	Perform Final Integration Testing	06-03-2026	07-03-2026	2	Ahsan, Abdullah
34	**Completion**	08-03-2026	14-03-2026	7	Ahsan, Abdullah
35	Finalize Documentation	08-03-2026	10-03-2026	3	Ahsan, Abdullah
36	Conduct Final Review	11-03-2026	12-03-2026	2	Ahsan, Abdullah
37	Archive Project Files	13-03-2026	13-03-2026	1	Ahsan, Abdullah
38	Project Closure	14-03-2026	14-03-2026	1	Ahsan, Abdullah

Table 5: WBS table

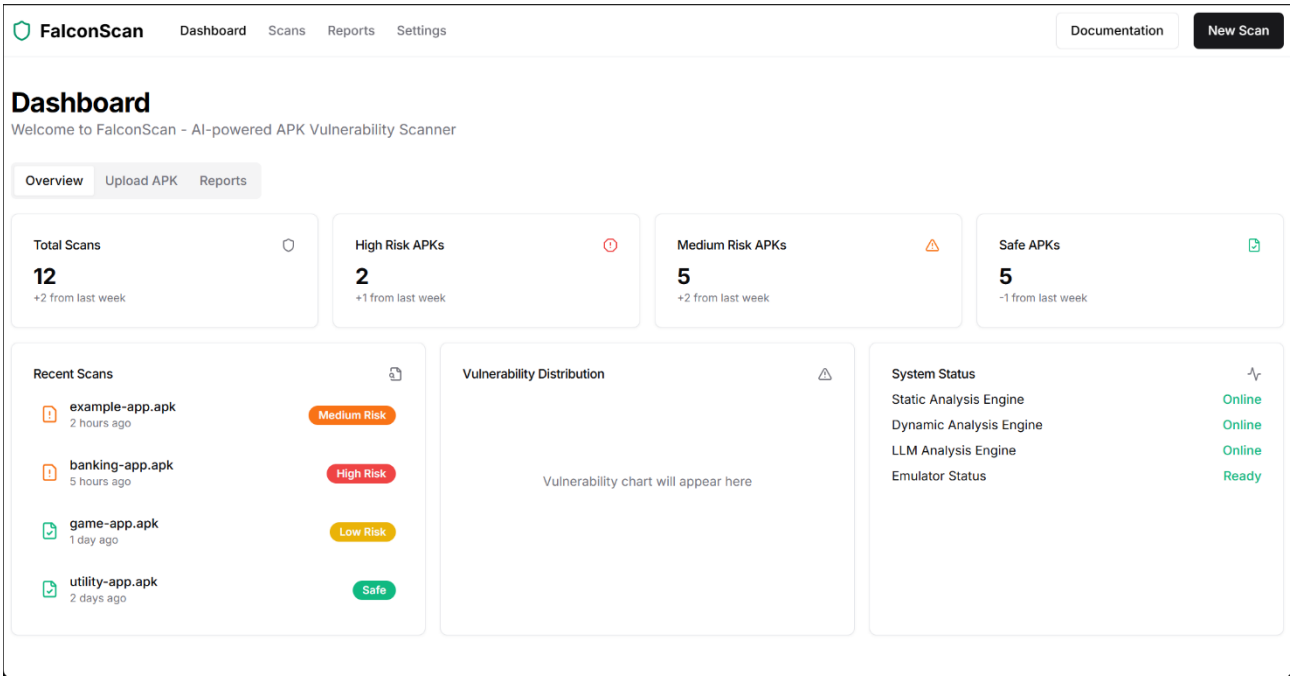


Act...	Resource	S...
ID	Task	D...
1	Analysis	14
2	Requirement Meetings	4
3	Communication with Stakeholders	3
4	Create Requirement Specification	5
5	Analysis Finished	2
6	Design	22
7	Software Design	12
8	Interface Design	4
9	Create Design Specifications	5
10	Design Finished	1
11	Development	49
12	APK Upload & Management Module	7
13	Static Analysis Module	21
14	Dynamic Analysis Module	14
15	AI-Based Detection Module	21
16	Report Generation Module	14
17	Admin Dashboard & User Interface	14
18	Development Finished	1
19	Testing	28
20	Prepare Test Cases	3
21	Perform Module Testing	10
22	Perform Integration Testing	7
23	Document Issues Found	5
24	Correct Issues Found	2
25	Testing Finished	1
26	Implementation	14
27	Deploy APK Upload & Management Module	3
28	Deploy Static Analysis Module	3
29	Deploy Dynamic Analysis Module	3
30	Deploy AI-Based Detection Module	3
31	Deploy Report Generation Module	3
32	Deploy Admin Dashboard & User Interface	3
33	Perform Final Integration Testing	2
34	Completion	7
35	Finalize Documentation	3
36	Conduct Final Review	2
37	Archive Project Files	1
38	Project Closure	1

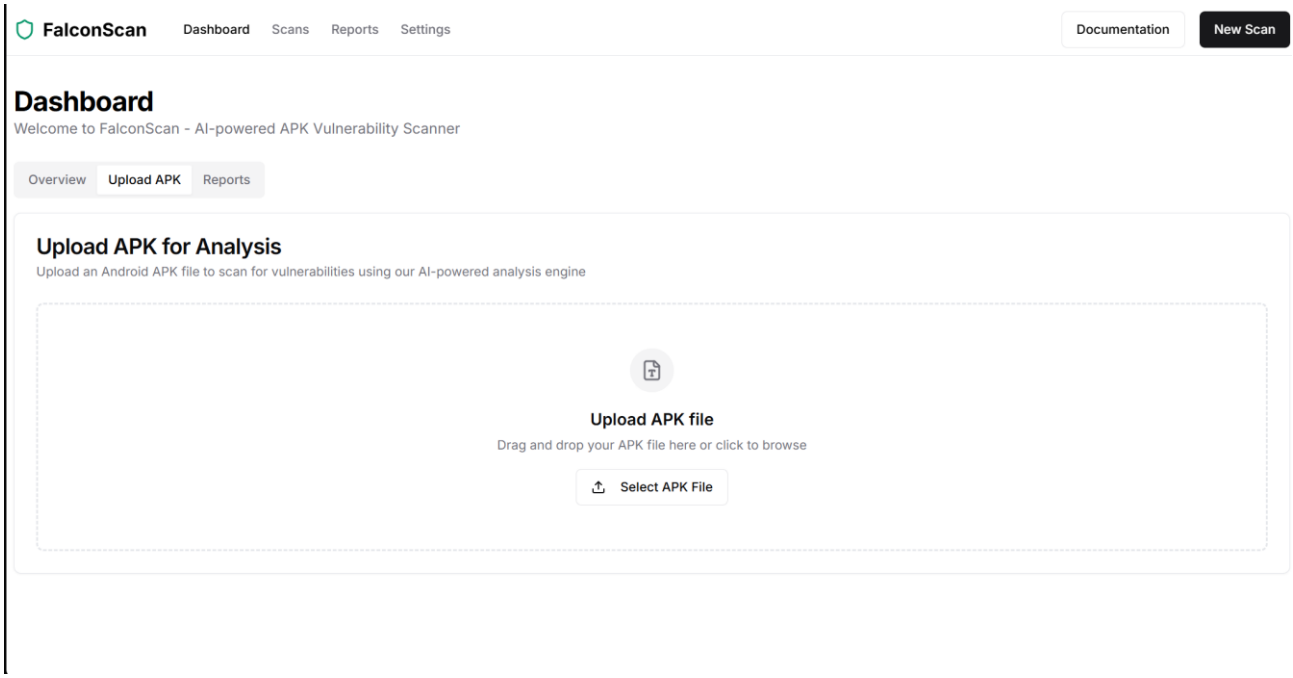
Table 6: Gantt Chart

14. Mockups

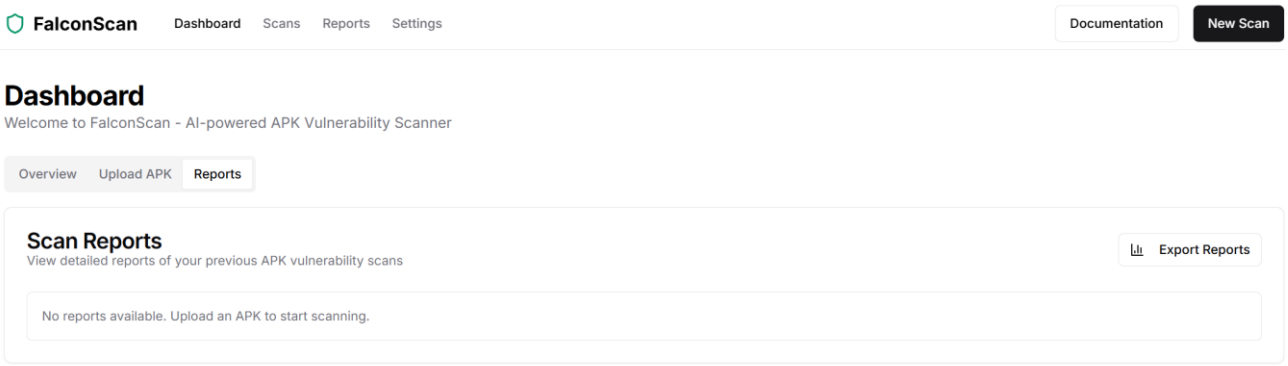
DASHBOARD



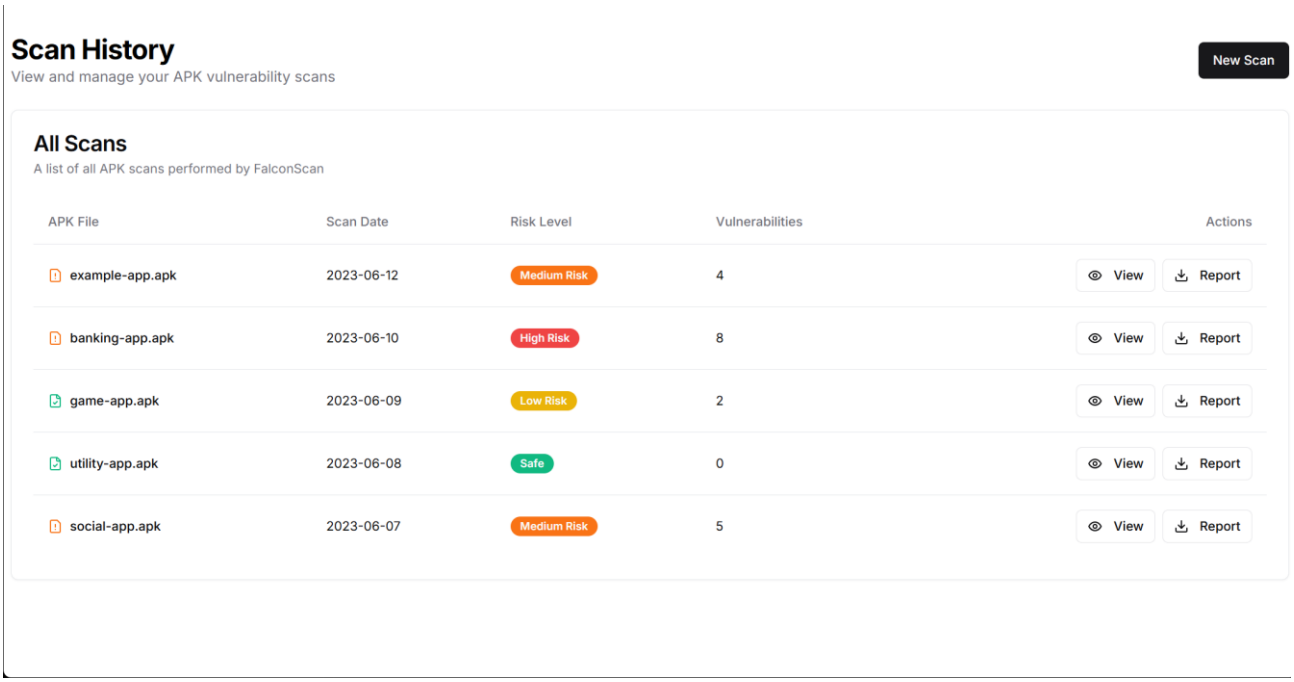
UPLOAD APK



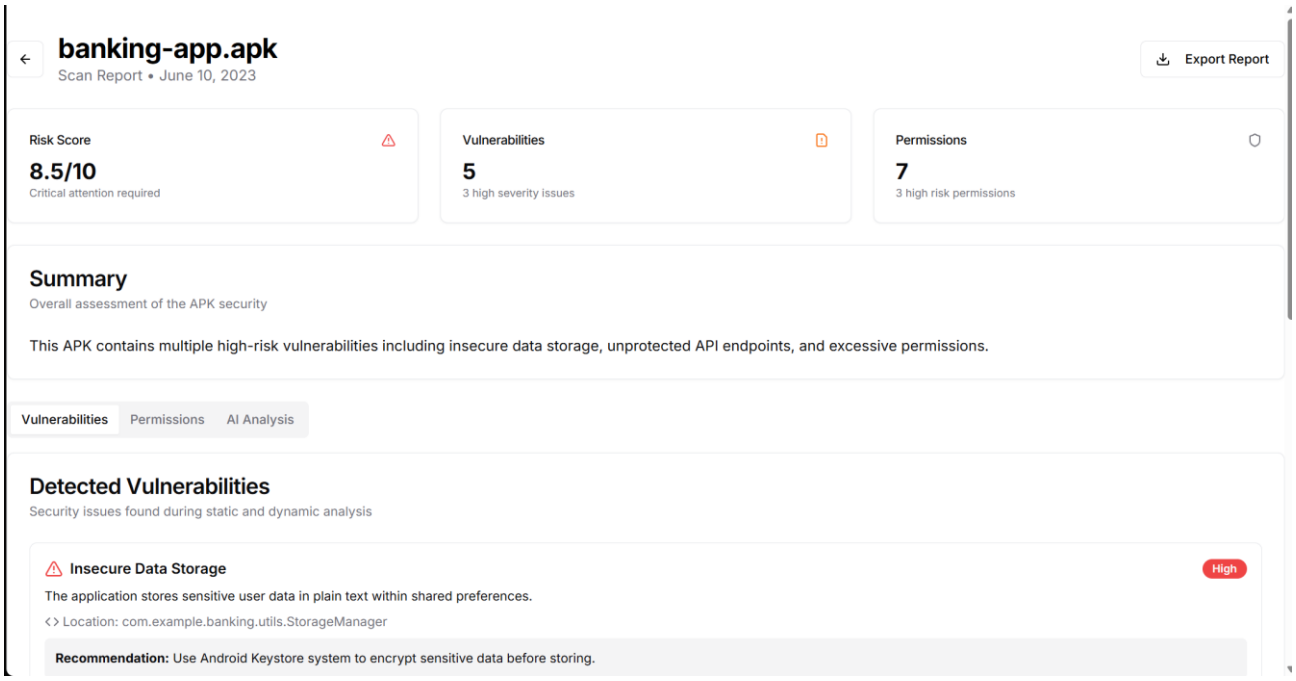
SCAN REPORTS



SCAN HISTORY



REPORT OF APK SCAN



15. References

1. *garak: A Framework for Security Probing Large Language Models*. Available at: https://github.com/NVIDIA/garak (Accessed: 5 May 2025).
2. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. Available at: https://arxiv.org/html/2406.18894v1 (Accessed: 12 April 2025).
3. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 22 April 2025).
4. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. ResearchGate. Available at: https://www.researchgate.net/publication/389809197_Assessing_the_Effectiveness_of_LLMs_in_Android_Application_Vulnerability_Analysis (Accessed: 30 April 2025).
5. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 17 May 2025).
6. *Vooki Android App Scanner*. Available at: https://www.vegabird.com/yaazhini/ (Accessed: 8 May 2025).
7. *Android Application Security Testing*. Available at: https://www.youtube.com/watch?v=PbvrxCvjsuY (Accessed: 2 May 2025).
8. *Android Penetration Testing with MobSF | OWASP ASST*. Available at: https://www.youtube.com/watch?v=qV_UolZEjuc (Accessed: 10 April 2025).
9. *Mobile App Testing and Security*. Available at: https://www.veracode.com/security/mobile-app-testing (Accessed: 25 April 2025).
10. *App Defense Alliance - MASA Requirements*. Available at: https://appdefensealliance.dev/masa/masa-requirements (Accessed: 19 May 2025).
11. *LLM Vulnerability Scanning Using Guardrails*. Available at: https://docs.nvidia.com/nemo/guardrails/latest/evaluation/llm-vulnerability-scanning.html (Accessed: 29 April 2025).
12. *Security in Mobile Applications: An Empirical Study on Android*. University of Padua. Available at: https://thesis.unipd.it/retrieve/56187322-2a56-4404-8537-626a447eb9ea/Braconaro_Elisa.pdf (Accessed: 14 April 2025).
13. *Vulnerability Scanners Topic*. Available at: https://github.com/topics/vulnerability-scanners (Accessed: 21 May 2025).
14. *SpringerOpen Article on Cybersecurity Topics*. Available at: https://cybersecurity.springeropen.com/articles/10.1186/s42400-025-00361-w (Accessed: 7 April 2025).
15. *Large Language Model (LLM) for Software Security Code Analysis, Malware Analysis & Reverse Engineering*. ResearchGate. Available at: https://www.researchgate.net/publication/390671720_Large_Language_Model_LLM_for_Software_Security_Code_Analysis_Malware_Analysis_Reverse_Engineering (Accessed: 27 May 2025).
16. *Electronics Journal - Recent Trends in Electronics and Electrical Engineering*. Available at: https://www.mdpi.com/2079-9292/13/23/4718 (Accessed: 3 May 2025).
17. *Vulnerability Detection in Recent Android Apps: An Empirical Study*. ResearchGate. Available at: https://www.researchgate.net/publication/345162631_Vulnerability_detection_in_recent_Android_apps_An_empirical_study (Accessed: 18 April 2025).
18. *Advances in Software Security and Machine Learning*. Available at: https://arxiv.org/html/2405.03644v2 (Accessed: 20 May 2025).
19. *garak: A Framework for Security Probing Large Language Models*. Available at: https://github.com/NVIDIA/garak (Accessed: 11 May 2025).
20. *Assessing the Effectiveness of LLMs in Android Application Vulnerability Analysis*. Available at: https://arxiv.org/html/2406.18894v1 (Accessed: 9 April 2025).

21. *Android-Security-Assessment-Tools*. Available at: https://github.com/rostomzed/Android-Security-Assessment-Tools (Accessed: 24 April 2025).
22. *Understanding the Impact of LLMs in Cybersecurity and Software Engineering*. arXiv preprint arXiv:2502.18474. Available at: https://arxiv.org/abs/2502.18474 (Accessed: 16 May 2025).

16. Plagiarism Report

turnitin Page 2 of 11 - AI Writing Overview Submission ID trn:oid:::30225:459197344

5% Similarity Detected

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a studentâ€™s work. We encourage you to learn more about Turnitinâ€™s AI detection capabilities before using the tool.

Detection Groups

0 AI-generated only 5%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitinâ€™s AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.



False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

turnitin Page 2 of 11 - AI Writing Overview Submission ID trn:oid:::30225:459197344